Real-world (part 2) — run in parallel, fall back, and stream — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — Three LCEL tools, one idea each
- Hint 2 — The plan, and almost the whole thing

Hint 1 — Three LCEL tools, one idea each

Make each one provable:

- Time the parallel step with `time.time()` before and after. Two functions that each `time.sleep(1)` should take about 1 second together, not 2.
- Fall back only on errors that pass on their own: `exceptions_to_handle=(openai.APIConnectionError, openai.RateLimitError)`. A `KeyError` from your own code must still raise.
- Force a real outage with no key and no cost: `ChatOpenAI(model="gpt-4o-mini", base_url="http://127.0.0.1:9", max_retries=0)` — nothing listens on port 9.
- Join the streamed pieces into one string, so the caller gets the full text too.

Hint 2 — The plan, and almost the whole thing

```
build_lookup_step() -> Runnable        # the RunnableParallel
with_backup(chain) -> Runnable         # .with_fallbacks(...)
stream_text(chain, inputs) -> str      # print pieces, return the joined text

main:
    case 1: keys, and "under 1.5 s?"
    case 2: outage -> backup used
    case 3: a buggy step -> KeyError still raised
    case 4: stream, then the full text's length
```

Write it, run it, then compare against the Solution.

Full solution: Show me the solution